

NYCU 2025 DCS 數位電路與系統 期末考

考試時間: 2025/05/27 15:30~17:20 請確實填寫姓名、學號與工作站帳號，題目卷會回收

姓名：

學號：

工作站帳號：

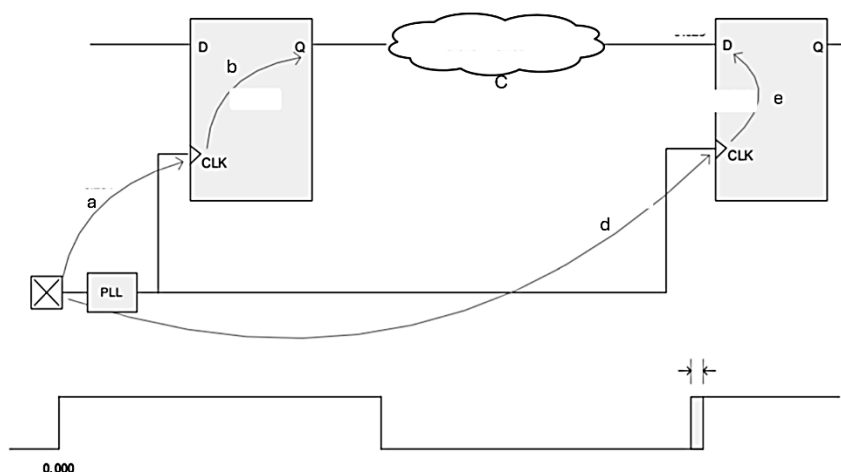
1. (10%) (Pipeline and parallel) Given there is a pipelined design with three stages, A, B, and C. The latency for A, B, and C are 8/16/4 cycles, respectively.
 - (a) (2%) Plot the pipeline diagram for four consecutive inputs. Indicate the hardware utilization rate for A, B, and C, respectively.
 - (b) (3%) How can this design be pipelined using the minimum number of stages while maximizing hardware utilization?
 - (c) (2%) Following (a), assume the B stage has variable delay controlled by an additional input: 16 cycles when the additional input is 1, and 4 cycles when it is 0. Plot the pipeline diagram for four consecutive inputs, where the additional input sequence is 1010.
 - (d) (3%) How to prevent the design in (c) from stall? Plot your block diagram.
2. (10%) For the following timing report, please draw its register-to-register timing path labeled from a to e in the figure. (Note. You can combine all combinational logic delay into one delay.)

你已經被期末好運貓貓造訪



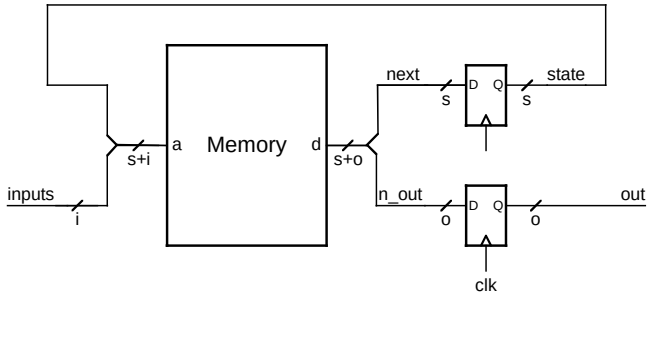
你可以分享這張圖並且打出
「感謝期末好運貓貓」
但分數並不會因此比較高

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (propagated)	0.15	0.15
reg_file/data_reg[15]/CK (DFCNQD1BWP7T30P140)	0.00	0.15 r
reg_file/data_reg[15]/Q (DFCNQD1BWP7T30P140)	0.12	0.27 r
reg_file/data_out[15] (reg_file)	0.00	0.27 r
U156/A1 (NAND2D1BWP7T30P140)	0.00	0.27 r
U156/Z (NAND2D1BWP7T30P140)	0.04	0.31 f
U157/A1 (INVD1BWP7T30P140)	0.01	0.32 f
U157/Z (INVD1BWP7T30P140)	0.02	0.34 r
alu_out_reg[7]/D (DFCNQD1BWP7T30P140)	0.00	0.34 r
data arrival time		0.34
clock clk (rise edge)	1.00	1.00
clock network delay (propagated)	0.18	1.18
clock uncertainty	-0.05	1.13
alu_out_reg[7]/CK (DFCNQD1BWP7T30P140)	0.00	1.13 r
library setup time	-0.04	1.09
data required time		1.09
data required time		1.09
data arrival time		-0.34
slack (MET)		0.75



3. (10%) (Microcode programming and FSM)

	$Q_1 Q_2 Q_3$	$Q_1^+ Q_2^+ Q_3^+$		Z	
		$X = 0$	$X = 1$	$X = 0$	$X = 1$
A	000	100	101	1	0
B	100	111	110	1	0
C	101	110	110	0	1
D	111	011	011	0	1
E	110	011	010	1	0
H	011	000	000	0	1
M	010	000	x x x	1	x
-	001	x x x	x x x	x	x



(a) (6%) Microcode is a commonly used method to implement FSM with ROM table. List the ROM contents for the following FSM table. You shall list the address and data of the ROM contents.

(b) (4%) The memory size could reach a large one if the state and input is increasing. Could you propose two methods to decrease the memory size?

4. (10%) (Memory usage) Assume you have a basic 1Kx8bits SRAM. For the following problem, plot the block diagram and specify your address and data bits clearly.

(a) (2%) Build a 1Kx32bits SRAM with the basic SRAM block.

(b) (3%) Build a 4Kx8bits SRAM with the basic SRAM block.

(c) (5%) The basic SRAM block only allow one read or one write. Build a memory system (2Kx8bits) with the basic SRAM block that allows two ports access (one read and one write per cycle or two read per cycle or two write per cycle. Assume the two ports access different addresses. There are two address spaces divided evenly, and the two ports won't access the same address space at the same time.) Hint. Use two memory blocks. Add additional logic if necessary. Explain how it works.

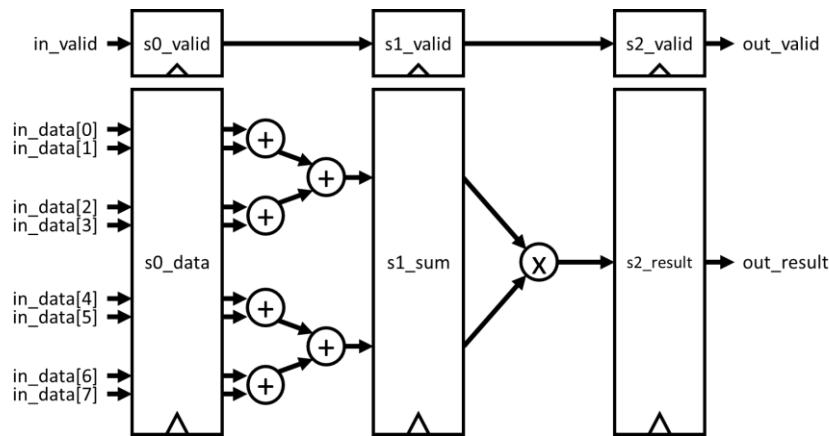
5. (10%) Metastability and Synchronizer.

(a) (3%) Metastability is a critical issue in clock domain crossing. What does metastability mean?

(b) (3%) Why can we use two flip-flops to prevent metastability during clock domain crossing?

(c) (4%) Why is Gray code a preferred method for transmitting multi-bit signals across clock domains?

6. (12%) Jerry designed a SystemVerilog circuit based on the block diagram below. The circuit takes eight 8-bit inputs, splits them into two groups of four, sums each group, and multiplies the two sums to produce the final output. He synthesized the design with a 4 ns clock, and the corresponding timing report is provided. Answer the following questions based on the diagram and report.



Point	Incr	Path
-----	-----	-----
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
s1_sum_reg_0__1_/CK (DFFRHQX4)	0.00	0.00 r
s1_sum_reg_0__1_/Q (DFFRHQX4)	0.31	0.31 r
U171/Y (BUF8)	0.16	0.48 r
U270/Y (XNOR2XL)	0.40	0.88 f
U202/Y (OAI22XL)	0.23	1.11 r
U528/CO (ADDFX1)	0.68	1.79 r
U217/S (ADDFHX1)	0.42	2.21 f
U529/Y (NOR2X1)	0.15	2.36 r
U530/Y (INVX2)	0.07	2.43 f
U543/Y (AOI21X1)	0.15	2.58 r
U544/Y (OAI21X1)	0.15	2.73 f
U546/Y (AOI21X2)	0.24	2.97 r
U554/Y (OAI21X2)	0.17	3.14 f
U573/Y (AOI21XL)	0.22	3.36 r
U578/Y (XOR2X1)	0.25	3.61 f
s2_result_reg_16_/D (DFFRHQXL)	0.00	3.61 f
data arrival time		3.61
clock clk (rise edge)	4.00	4.00
clock network delay (ideal)	0.00	4.00
clock uncertainty	-0.10	3.90
s2_result_reg_16_/CK (DFFRHQXL)	0.00	3.90 r
library setup time	-0.28	3.62
data required time		3.62
data required time		3.62
data arrival time		-3.61
slack (MET)		0.01

(a) (3%) The timing report shows a slack of 0.01 ns. Does this mean the minimum clock period is 3.99 ns?

Explain.

(b) (3%) Jerry considers inserting a pipeline stage into the adder tree that connects stage 0 and stage 1 to reduce the minimum clock period. Would this be effective? **Why or why not?**

(c) (3%) Jerry wrote a test task shown below to verify the reset signal. Are there any issues with it? If so, **describe when it may fail and provide a corrected version.**

```
task check_reset; begin
    force clk = 0;
    rst_n = 'b0;
    #(20) if(out_valid != 'b0 || out_result != 'b0) reset_fail;
    rst_n = 'b1;
    #(10); release clk;
end endtask
```

(d) (3%) Jerry wants to add a new 2-bit input signal in_mode, which specifies the offset for shifting in_data.

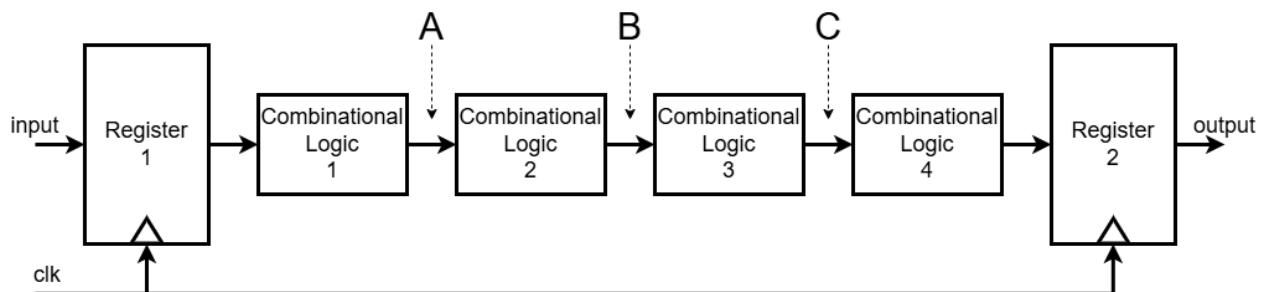
For example, if in_mode = 2, in_data is circularly shifted by 2 positions. That is:

$$\text{out_result} = (\text{in_data}[6] + \text{in_data}[7] + \text{in_data}[0] + \text{in_data}[1]) * (\text{in_data}[2] + \text{in_data}[3] + \text{in_data}[4] + \text{in_data}[5])$$

To implement this, both Jerry and his friend, Tom, modified the signal s1_sum_ns, the D input of flip-flop s1_sum. Based on their implementations, which design results in a smaller area? **Explain your reasoning.**

Tom's design:	Jerry's design:
<pre> logic [0:7][7:0] select; always_comb begin case(s0_mode) 2'b00: select = s0_data; 2'b01: select = {s0_data[7], s0_data[0:6]}; 2'b10: select = {s0_data[6:7], s0_data[0:5]}; 2'b11: select = {s0_data[5:7], s0_data[0:4]}; default: select = 'bx; endcase s1_sum_ns[0] = select[0] + select[1] + select[2] + select[3]; s1_sum_ns[1] = select[4] + select[5] + select[6] + select[7]; end </pre>	<pre> always_comb begin case(s0_mode) 2'b00: begin s1_sum_ns[0] = s0_data[0] + s0_data[1] + s0_data[2] + s0_data[3]; s1_sum_ns[1] = s0_data[4] + s0_data[5] + s0_data[6] + s0_data[7]; end 2'b01: begin s1_sum_ns[0] = s0_data[7] + s0_data[0] + s0_data[1] + s0_data[2]; s1_sum_ns[1] = s0_data[3] + s0_data[4] + s0_data[5] + s0_data[6]; end 2'b10: begin s1_sum_ns[0] = s0_data[6] + s0_data[7] + s0_data[0] + s0_data[1]; s1_sum_ns[1] = s0_data[2] + s0_data[3] + s0_data[4] + s0_data[5]; end 2'b11: begin s1_sum_ns[0] = s0_data[5] + s0_data[6] + s0_data[7] + s0_data[0]; s1_sum_ns[1] = s0_data[1] + s0_data[2] + s0_data[3] + s0_data[4]; end default: s1_sum_ns = 'bx; endcase end </pre>

7. (12%) Pipeline.



In the circuit diagram above, Combinational Logic 1~4 are all combinational circuit blocks, and Register 1 and 2 are positive-edge-triggered D flip-flops. The delays of Combinational Logic 1~4 are 3ns, 5ns, 7ns, and 1ns respectively. Assume that other delays are negligible, and **the register setup time is 1ns**, hold time is 0.

- (2%) If you want to insert a pipeline register at one of the locations A, B, or C, which location would be the best choice? **Please explain why.**
- (2%) Based on the pipeline you made in part (a), what is the shortest possible clock cycle time that will not cause a setup time violation?
- (2%) Calculate the throughput before and after pipelining, pipelining is based on your choice in part (a). Assume only one output per cycle, and the cycle time is the shortest possible without causing a timing violation. (Hint: Cycle time might be different before and after pipelining.)
- (3%) Will the latency be shortened after inserting the pipeline register as described in part (a)? Explain your reasoning. Assume that the cycle time is the shortest possible without causing a timing violation.
- (3%) Based on the pipeline you made in part (a), is it possible to further improve the throughput by adding another pipeline register at one of the locations A, B, or C? **Please explain why.**

8. (14%) Waveform.

Please refer to the following Pattern and Design code and draw the corresponding waveform diagram.

Please complete the waveform on the next page of this exam paper.

```

11 //-----
12 //  PATTERN
13 //-----
14 output logic clk, rst_n;
15 output logic in_valid;
16 output logic [3:0] in_num;
17 input out_valid;
18 input [3:0] out_num;
19
20 parameter CYCLE = 10;
21 always #(CYCLE/2.0) clk = ~clk;
22 initial clk = 0;
23
24 initial begin
25     in_valid = 0;
26     in_num = 0;
27     rst_n = 1;
28     @(negedge clk) rst_n = 0;
29     @(negedge clk) rst_n = 1;
30     in_valid = 1;
31     in_num = 8;
32     @(negedge clk);
33     in_num = 5;
34     @(negedge clk);
35     in_valid = 0;
36     in_num = 0;
37     wait(out_valid == 1);
38     repeat(2) @(negedge clk);
39     in_valid = 1;
40     in_num = 2;
41     @(negedge clk);
42     in_valid = 0;
43     in_num = 0;
44
45     repeat(10) @(negedge clk);
46     $finish;
47 end

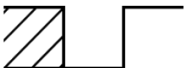
```

```

11 //-----
12 //  DESIGN
13 //-----
14 input clk, rst_n;
15 input in_valid;
16 input [3:0] in_num;
17 output logic out_valid;
18 output logic [3:0] out_num;
19
20 parameter IDLE=0, DIN=1, COM1=2, COM2=3, OUT=4;
21 logic [2:0] cs, ns;
22 logic [3:0] out_num_next;
23 logic signed [3:0] in_reg, in_reg_next;
24
25 always ff @(posedge clk) begin
26     if(!rst_n) cs <= IDLE;
27     else cs <= ns;
28 end
29 always_comb begin
30     case(cs)
31     IDLE:
32         if(in_valid) ns = DIN;
33         else ns = IDLE;
34     DIN:
35         if(!in_valid) ns = OUT;
36         else if(in_reg < 4) ns = COM1;
37         else ns = COM2;
38     COM1: ns = OUT;
39     COM2: ns = COM1;
40     OUT: ns = IDLE;
41     default: ns = 'dx;
42     endcase
43 end
44
45 always ff @(posedge clk) begin
46     if(!rst_n) in_reg <= 0;
47     else in_reg <= in_reg_next;
48 end
49 always_comb begin
50     if(in_valid) in_reg_next = in_num;
51     else in_reg_next = in_reg;
52 end
53
54 always ff @(posedge clk) begin
55     if(!rst_n) out_num <= 0;
56     else out_num <= out_num_next;
57 end
58 always_comb begin
59     if(cs == COM1) out_num_next = out_num + 2;
60     else if(cs == COM2) out_num_next = out_num >> 1;
61     else if(in_valid) out_num_next = in_num;
62     else out_num_next = in_reg;
63 end
64 assign out_valid = (cs == OUT)? 1: 0;

```

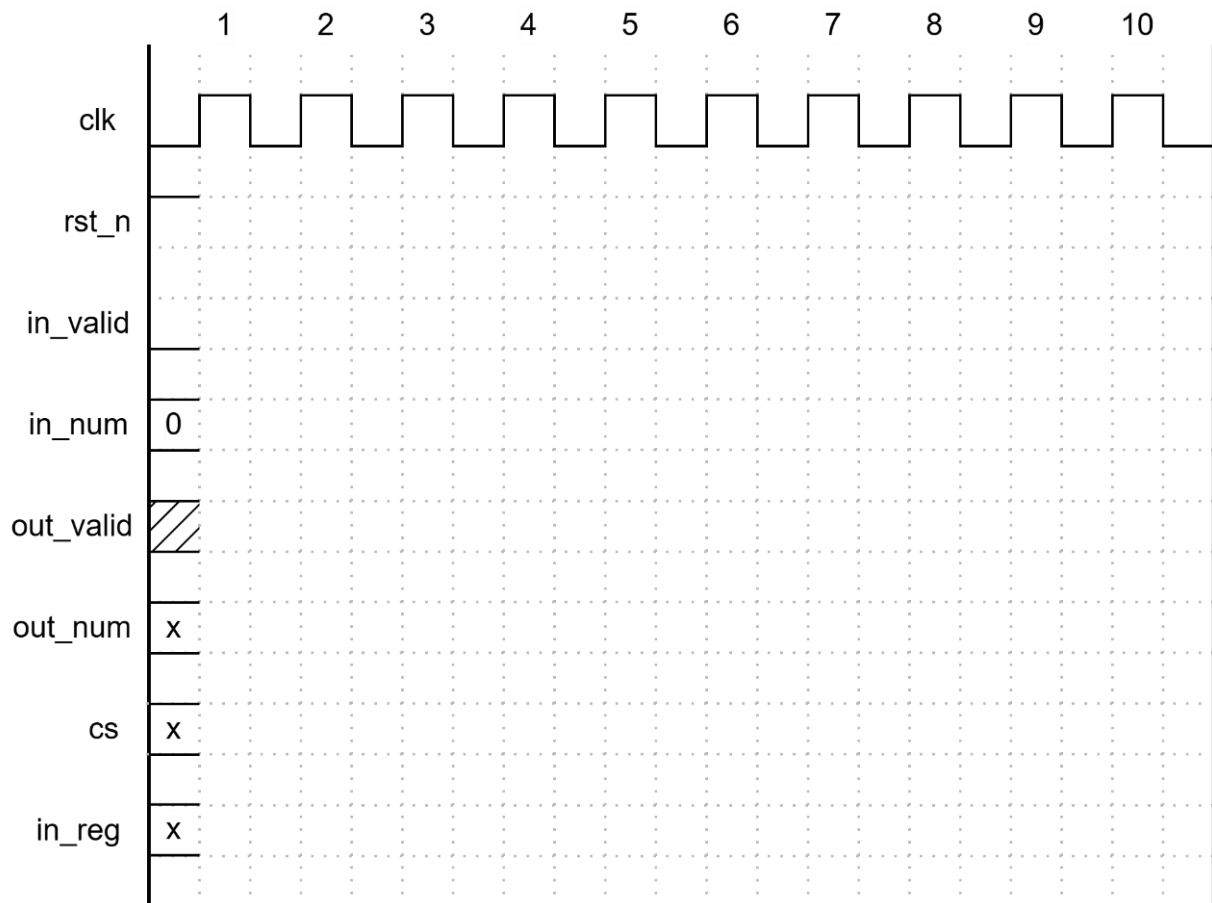
1-bit
signal



multi-bit
signal



The examples of drawing 1-bit and multi-bit signals are shown above. Please draw the waveforms for the signals `rst_n`, `in_valid`, `in_num`, `out_valid`, `out_num`, `cs`, and `in_reg` for at least 10 clock cycles. All the multi-bit signals should be expressed in decimal. **Note that the “`in_reg`” signal should be represented in signed 2’s complement format** (e.g., mark -1 on the waveform if `in_reg = 4'b1111`), while all other signals are assumed to be unsigned.



9. (12%) For Flip-Flop A in Table 1 (used for DFF X, Y, Z), and a 2 GHz clock, check the Fig. 1 (the symbol “ t_c ” represents the logic contamination delay, while “ t_d ” represents the logic propagation delay) for timing violations.
- (a) (4%) Is there any setup/hold time violation? If so, indicate where it happens along with the timing check formula.
- (b) (4%) How to solve it? And write the timing check formula after your modification.
- (c) (4%) Recheck for timing violations. Is it possible to run faster than 2 GHz clock? Derive the cycle time for this maximum clock frequency.

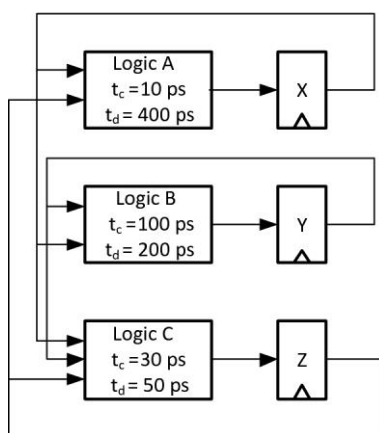


Fig. 1

Parameter (ps)	DFF A
Setup time: t_{setup}	30
Hold time: t_{hold}	35
Contamination delay, clock to q, t_{ccq}	10
Propagation delay, clock to q, t_{pcq}	20

Table 1

(Hint: Setup time check, $T_c \geq t_{\text{pcq}} + t_{\text{pd}} + t_{\text{setup}}$; Hold time check, $t_{\text{cd}} + t_{\text{ccq}} \geq t_{\text{hold}}$)